



BIRZEIT UNIVERSITY

Department of Electrical and Computer Engineering

ENCS5141 - AI Lab

Summer Semester 2025–2026

Arabic Sentiment Analysis

A Comparative Study of Representations and Classifiers on ArSarcasm-v2

Student: Abedalrheem Fialah
Student ID: 1220216
Instructor: Aziz Qaroush
Assignment: Second Assignment

September 23, 2026

Abstract

This report presents a controlled study of three-class Arabic sentiment classification using ArSarcasm-v2. The work compares TF-IDF, Word2Vec, and FastText representations with Naive Bayes, Random Forest, convolutional neural network (CNN), vanilla recurrent neural network (RNN), and long short-term memory (LSTM) classifiers. A deterministic, leakage-safe 60/20/20 split is constructed after resolving duplicate-text conflicts. All data-dependent resources are fitted on the training partition, hyperparameters are selected using validation macro F1, and the test partition is evaluated only after selection is frozen. The study also includes one-stage-at-a-time preprocessing ablation, Arabic-adapted Easy Data Augmentation (EDA), balanced class weighting, three-seed stability analysis, and auxiliary error analysis by sarcasm and dialect. The validation-selected FastText CNN achieves a seed-42 test macro F1 of 0.6380 and accuracy of 0.6674. TF-IDF with Multinomial Naive Bayes is a very close, substantially simpler baseline at 0.6348 test macro F1. Balanced class weighting improves positive-class F1 for the controlled Random Forest experiment, whereas the tested EDA policy reduces it. These results show that subword-aware sequence representations are competitive, but that a carefully tuned sparse lexical baseline remains unusually strong on this dataset.

Contents

1	Problem Formulation and Project Overview	1
2	Dataset and Arabic Sentiment Challenges	1
2.1	ArSarcasm-v2	1
2.2	Duplicate and conflict policy	1
2.3	Arabic social-media challenges	2
3	Methodology	2
3.1	Arabic preprocessing pipeline	2
3.2	Feature representations	3
3.2.1	TF-IDF	3
3.2.2	Word2Vec	3
3.2.3	FastText	3
3.3	Classification models	3
3.3.1	Traditional models	3
3.3.2	CNN architecture	3
3.3.3	Vanilla RNN architecture	4
3.3.4	LSTM architecture	4
3.4	Training and systematic hyperparameter optimization	4
3.5	Controlled preprocessing, augmentation, and imbalance studies	4
3.6	Evaluation metrics	5
4	Experimental Results	5
4.1	Full representation-model comparison	5

4.2	Selected-model confusion matrix and seed stability	6
4.3	Preprocessing ablation	6
4.4	Arabic data augmentation	7
4.5	Class-imbalance handling	7
4.6	Influence of hyperparameter tuning	8
5	Comparative Analysis and Discussion	8
5.1	Impact of representation choice	8
5.2	Traditional versus deep learning	8
5.3	Non-sequential versus sequential neural models	9
5.4	Strengths and limitations of each combination	9
5.5	Sarcasm and dialect error analysis	10
6	Challenges Encountered and Mitigation Strategies	10
7	Reproducibility and Implementation	11
8	Conclusion	11
A	All Test Confusion Matrices	12
B	Search Spaces	13

List of Tables

1	Leakage-safe sentiment-stratified grouped split.	2
2	Validation-selected model hyperparameters.	4
3	Validation selection and frozen test results for all required models.	5
4	FastText CNN fixed-configuration test results across seeds.	6
5	Preprocessing ablation on validation data.	7
6	Validation effect of positive-class Arabic EDA.	7
7	Validation effect of balanced class weights on TF-IDF Random Forest.	7
8	Validation macro-F1 gain from the first configured trial to the selected trial.	8
9	Qualitative strengths and limitations of the evaluated combinations.	9
10	Selected-model test performance by sarcasm label.	10
11	Selected-model test performance by dialect.	10
12	Test confusion matrices for all 12 representation-model combinations.	12
13	Systematic validation search spaces.	13

List of Figures

1	Validation and test macro F1 for all required combinations. FT denotes FastText and W2V denotes Word2Vec.	5
2	FastText CNN test confusion matrix at seed 42.	6

1 Problem Formulation and Project Overview

The task is to learn a classifier

$$f : \mathcal{X} \rightarrow \{\text{NEG}, \text{NEU}, \text{POS}\},$$

where $\mathcal{X}$ is the space of Arabic social-media posts. The central research question is how the choice of representation and classifier changes performance under a shared, leakage-safe experimental protocol. Three representation families are compared: sparse TF-IDF features, dense Word2Vec embeddings, and subword-aware FastText embeddings. Traditional models are compared with a non-sequential CNN and the sequential vanilla RNN and LSTM architectures.

The project additionally asks whether individual preprocessing decisions, minority-class data augmentation, and class weighting materially change performance. These questions are evaluated as controlled validation studies, not folded into an unconstrained search for a single favorable test score. Macro F1 is the primary selection metric because positive sentiment is the minority class. Accuracy, macro and weighted precision, recall, F1, per-class metrics, and confusion matrices are retained for comprehensive evaluation.

Transformer embeddings are optional in the assignment and are not included. The mandatory TF-IDF, Word2Vec, and FastText comparison is completed in full, allowing experimental effort to focus on rigorous controls and interpretable comparisons.

2 Dataset and Arabic Sentiment Challenges

2.1 ArSarcasm-v2

ArSarcasm-v2 was released for the WANLP 2021 shared task on Arabic sarcasm and sentiment detection [1]. It contains 15,548 tweets: 12,548 in the published training file and 3,000 in the published testing file. Every record used here has four fields: tweet text, sentiment, sarcasm, and dialect. Sentiment is the only prediction target. Sarcasm and dialect are reserved for descriptive error analysis.

The source files are downloaded from the official repository at revision `eb85513e133cfc67e30aca4968fadbe2594fcb42`. Their SHA-256 checksums are verified before loading. This protects the experiment from silent changes to the source data.

2.2 Duplicate and conflict policy

Combining both published files reveals exact and normalized-text duplicates, including groups with contradictory sentiment annotations. The policy is:

1. normalize whitespace to identify equivalent raw texts;
2. remove all 52 rows in the 26 groups with conflicting sentiment labels;
3. collapse each remaining duplicate group to one representative;
4. resolve auxiliary sarcasm and dialect disagreement by a deterministic mode rule; and
5. split by the complete default processed representation so equivalent model inputs cannot cross partitions.

This retains 15,455 records. Five-fold stratified group assignment is used: folds 0–2 form training, fold 3 validation, and fold 4 testing. Table 1 summarizes the resulting 60/20/20 partition.

Table 1: Leakage-safe sentiment-stratified grouped split.

Partition	NEG	NEU	POS	Total	Share
Training	3,750	3,876	1,647	9,273	60.0%
Validation	1,249	1,292	550	3,091	20.0%
Test	1,249	1,293	549	3,091	20.0%
Total	6,248	6,461	2,746	15,455	100.0%

The training distribution is 40.44% NEG, 41.80% NEU, and 17.76% POS. Sarcasm represents 19.13% of training records. MSA represents 69.78%, while Maghrebi contains only 30 training examples. These skews motivate macro F1, minority-specific reporting, and caution in subgroup interpretation.

2.3 Arabic social-media challenges

Arabic sentiment analysis must handle orthographic variants, optional diacritics, elongation, rich morphology, informal spelling, dialect mixing, code-like social-media tokens, and sparse minority classes. Emoji and hashtags often carry direct sentiment. Negation can reverse polarity and should not be treated as an ordinary stop word. Sarcasm is also correlated with negative sentiment, so high subgroup accuracy can conceal failure on non-negative sarcastic examples. FastText is especially relevant because character n-grams can represent unseen inflected or misspelled forms [3].

3 Methodology

3.1 Arabic preprocessing pipeline

The pipeline is implemented as independently switchable stages so that each decision can be ablated without changing the others. It performs:

1. HTML entity decoding and HTML tag removal;
2. URL removal and mention replacement;
3. emoji demojization into stable semantic tokens;
4. hashtag unpacking while retaining the hashtag content;
5. conservative Arabic character normalization;
6. diacritic and tatweel removal;
7. digit normalization with a protected placeholder;
8. punctuation and unnecessary symbol removal;
9. repeated-character collapse;
10. NLTK Arabic stop-word removal; and
11. Qalsadi lemmatization.

Negations, mention and number placeholders, hashtag content, and emoji tokens are protected from inappropriate stop-word removal or morphological reduction. Taa marbuta is intentionally unchanged because mapping it to haa may conflate different words. ISRI stemming is available but disabled; lemmatization is the default less aggressive morphological choice. Four records require a conservative non-empty fallback. The final corpus contains 15,036 unique model inputs and no processed-text group crosses a partition boundary.

3.2 Feature representations

3.2.1 TF-IDF

Word-level unigrams and bigrams are transformed using sublinear term frequency, smoothed inverse document frequency, and ℓ_2 normalization. The vectorizer uses `min_df=2`, `max_df=0.98`, and at most 50,000 features. Its vocabulary and IDF statistics are fitted only on training text. The sparse representation is suitable for Multinomial Naive Bayes and Random Forest but is not passed to CNN, RNN, or LSTM models: vocabulary dimensions do not form a linguistic token sequence.

3.2.2 Word2Vec

Skip-gram Word2Vec [2] is trained on tokenized training text with dimension 200, context window 5, minimum count 2, 10 negative samples, and 20 epochs. A single worker and stable token hash improve repeatability. Mean document embeddings support Gaussian Naive Bayes and Random Forest. Token embeddings initialize the neural sequence models. Validation and test tokens never update the learned vectors.

3.2.3 FastText

FastText shares the Word2Vec settings and adds character n-grams of lengths 3–6 in a 200,000-entry bucket. Its subword construction can infer vectors for held-out word forms from training-derived character n-grams [3]. Mean document vectors support traditional classifiers; token projections support CNN, RNN, and LSTM models.

3.3 Classification models

3.3.1 Traditional models

Multinomial Naive Bayes is used only with non-negative TF-IDF features. Gaussian Naive Bayes is used for dense mean Word2Vec and FastText vectors. Random Forest [4] is evaluated with all three compatible fixed-width representations. Baseline forests are unweighted so the later class-weighting study isolates the cost-sensitive change.

3.3.2 CNN architecture

Each tweet is encoded to at most 64 tokens. Index 0 is padding; an explicit out-of-vocabulary vector is used when required. Frozen 200-dimensional training-only embeddings feed parallel one-dimensional convolutions with kernel sizes 3, 4, and 5. Global max pooling extracts the strongest feature from each filter, followed by dropout and a three-unit linear output layer. This follows the effective sentence-CNN design of Kim [5] while keeping representation learning separate from classifier learning.

3.3.3 Vanilla RNN architecture

The vanilla RNN uses a single tanh recurrent layer over the same frozen embeddings. True retained lengths select the final non-padding hidden state. The state passes through dropout and a three-class linear layer. This model is the sequential baseline and can capture order, but its ungated recurrence is more vulnerable to vanishing gradients and noisy long dependencies.

3.3.4 LSTM architecture

The LSTM replaces the tanh recurrent cell with gated memory [6]. The final valid hidden state is passed through dropout and a three-class output layer. It has greater capacity to retain longer-range information, at the cost of more parameters and training time.

3.4 Training and systematic hyperparameter optimization

All candidates are fitted on training data and ranked by validation macro F1. Weighted F1, accuracy, and a deterministic trial order break ties. No test metric participates in selection. Neural models use batch size 128, AdamW, weight decay 10^{-4} , unweighted cross-entropy, at most 10 epochs, patience 2, and validation macro F1 checkpointing. Table 2 lists every selected configuration.

Table 2: Validation-selected model hyperparameters.

Model	Selected hyperparameters	Epoch
TF-IDF + Multinomial NB	$\alpha = 0.5$, uniform class prior	—
TF-IDF + Random Forest	200 trees, no depth limit, min leaf 1	—
Word2Vec + Gaussian NB	variance smoothing 10^{-3}	—
Word2Vec + Random Forest	200 trees, depth 40, min leaf 1	—
Word2Vec + CNN	128 filters, dropout 0.5, learning rate 10^{-3}	4
Word2Vec + vanilla RNN	hidden 128, dropout 0.3, learning rate 10^{-3}	4
Word2Vec + LSTM	hidden 64, dropout 0.5, learning rate 10^{-3}	8
FastText + Gaussian NB	variance smoothing 10^{-6}	—
FastText + Random Forest	200 trees, no depth limit, min leaf 2	—
FastText + CNN	128 filters, dropout 0.3, learning rate 10^{-3}	5
FastText + vanilla RNN	hidden 128, dropout 0.3, learning rate 10^{-3}	4
FastText + LSTM	hidden 64, dropout 0.3, learning rate 10^{-3}	2

3.5 Controlled preprocessing, augmentation, and imbalance studies

The preprocessing ablation fixes TF-IDF and the selected Multinomial Naive Bayes parameters, then disables exactly one enabled preprocessing stage at a time. Every ablated vectorizer is re-fitted on its corresponding training text.

The augmentation study applies deterministic Arabic-adapted EDA to POS training records until POS matches the largest training class. Random swap and single-token deletion protect negations, emoji, hashtags, mentions, URLs, and placeholders. Unchanged outputs, training duplicates, synthetic duplicates, and processed-text collisions with validation or test are rejected. This adds 2,229 unique records and increases the training set from 9,273 to 11,502.

The imbalance study is kept separate from augmentation. Balanced class weights are calculated from training labels and applied to the otherwise unchanged selected TF-IDF Random Forest. This directly compares cost-sensitive and unweighted learning.

3.6 Evaluation metrics

For each class c , precision, recall, and F1 are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}.$$

Macro F1 is the unweighted mean across NEG, NEU, and POS; weighted F1 weights each class by support. Accuracy is the fraction of correct predictions. Fixed label ordering is used in every confusion matrix.

4 Experimental Results

4.1 Full representation-model comparison

Table 3 reports all mandatory combinations. Models are ordered by validation macro F1, the criterion fixed before test access. Macro precision and recall make the effect of class imbalance visible; weighted F1 is included for completeness.

Table 3: Validation selection and frozen test results for all required models.

Rank	Representation + classifier	Val. F1	Acc.	Macro P	Macro R	Macro F1	Wtd. F1	NEG F1	NEU F1	POS F1
1	FastText + CNN	0.6335	0.6674	0.6524	0.6313	0.6380	0.6642	0.6997	0.6886	0.5258
2	TF-IDF + Multinomial NB	0.6335	0.6671	0.6457	0.6312	0.6348	0.6641	0.7086	0.6870	0.5088
3	Word2Vec + LSTM	0.6296	0.6555	0.6361	0.6224	0.6262	0.6527	0.6947	0.6721	0.5117
4	Word2Vec + CNN	0.6256	0.6622	0.6477	0.6155	0.6254	0.6567	0.6858	0.6985	0.4919
5	Word2Vec + vanilla RNN	0.6177	0.6454	0.6270	0.6139	0.6156	0.6422	0.6928	0.6534	0.5005
6	FastText + LSTM	0.6164	0.6522	0.6353	0.6107	0.6189	0.6480	0.6839	0.6785	0.4943
7	FastText + vanilla RNN	0.6149	0.6509	0.6279	0.6074	0.6139	0.6464	0.6884	0.6790	0.4742
8	FastText + Random Forest	0.5986	0.6707	0.6812	0.6023	0.6128	0.6565	0.7089	0.7040	0.4253
9	Word2Vec + Random Forest	0.5900	0.6587	0.6568	0.5893	0.5978	0.6443	0.6955	0.6989	0.3990
10	TF-IDF + Random Forest	0.5886	0.6457	0.6518	0.5829	0.5948	0.6338	0.6667	0.6895	0.4282
11	Word2Vec + Gaussian NB	0.5279	0.5804	0.5929	0.5480	0.5435	0.5674	0.6483	0.5443	0.4378
12	FastText + Gaussian NB	0.5054	0.5675	0.5961	0.5327	0.5248	0.5482	0.6438	0.5099	0.4207

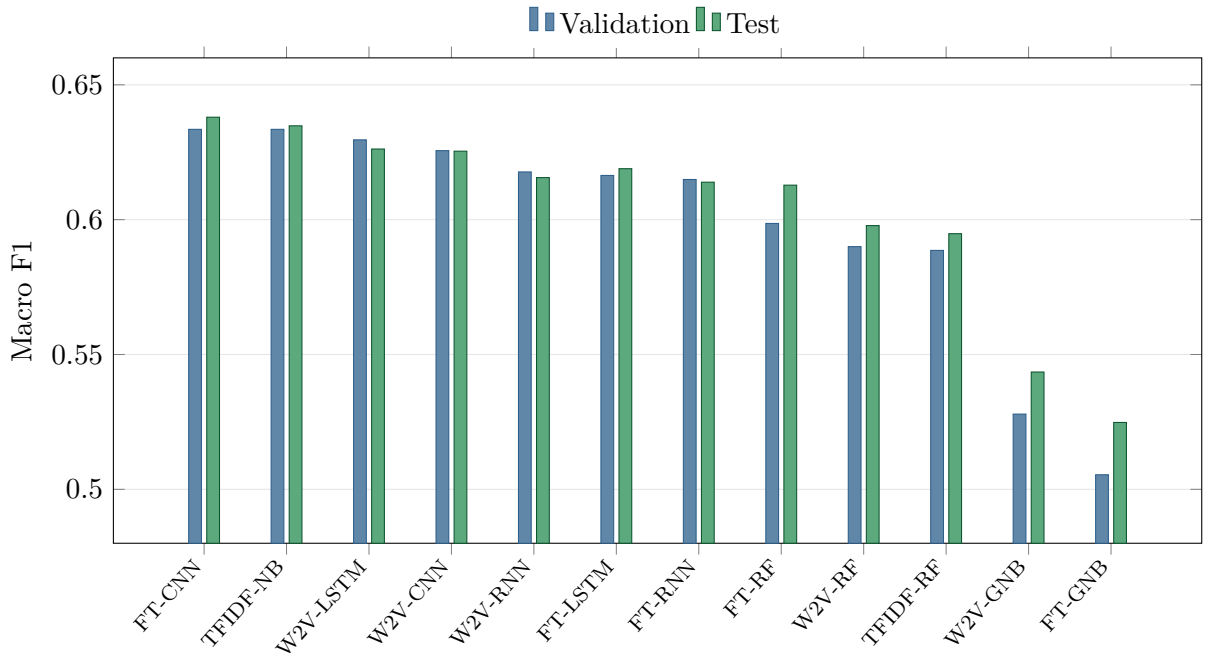


Figure 1: Validation and test macro F1 for all required combinations. FT denotes FastText and W2V denotes Word2Vec.

FastText CNN ranks first by validation macro F1, but its validation advantage over TF-IDF Multinomial NB is only 0.00003. On test data, the CNN advantage is 0.0032 macro-F1 points. This is evidence that subword-aware local sequence features help, but also that the sparse lexical baseline is highly competitive and much cheaper to train.

The highest test accuracy belongs to FastText Random Forest (0.6707), yet its macro F1 is only 0.6128 because positive F1 falls to 0.4253. This divergence illustrates why accuracy alone is insufficient under class imbalance.

4.2 Selected-model confusion matrix and seed stability

Actual	NEG	945	227	77
	NEU	332	858	103
	POS	175	114	260
		NEG	NEU	POS
		Predicted		

Figure 2: FastText CNN test confusion matrix at seed 42.

The selected model correctly classifies 260 of 549 positive examples. Most positive errors become NEG (175) rather than NEU (114), indicating that minority-class confusion is not simply uncertainty around neutrality. NEG and NEU have similar F1 scores, 0.6997 and 0.6886, while POS remains lower at 0.5258.

The validation-selected architecture and five-epoch training schedule are repeated under seeds 42, 1337, and 2026. The test macro-F1 mean is 0.6103 with sample standard deviation 0.0247. Positive F1 is more variable, with mean 0.4636 and standard deviation 0.0621.

Table 4: FastText CNN fixed-configuration test results across seeds.

Seed	Accuracy	Macro P	Macro R	Macro F1	POS F1
42	0.6674	0.6524	0.6313	0.6380	0.5258
1337	0.6503	0.6597	0.5805	0.5905	0.4015
2026	0.6457	0.6586	0.5928	0.6024	0.4635
Mean	0.6545	0.6569	0.6015	0.6103	0.4636
SD	0.0114	0.0039	0.0265	0.0247	0.0621

4.3 Preprocessing ablation

Table 5 reports one-stage-at-a-time changes relative to the full TF-IDF + Multinomial NB pipeline. A negative delta means the removed stage was beneficial.

Table 5: Preprocessing ablation on validation data.

Removed stage	Changed train rows	Macro F1	Δ Macro F1	POS F1	Δ POS F1
None (full pipeline)	0	0.6335	0.0000	0.5138	0.0000
Decode HTML entities	3	0.6335	0.0000	0.5138	0.0000
Remove HTML tags	0	0.6335	0.0000	0.5138	0.0000
Remove URLs	3,102	0.6381	+0.0046	0.5116	-0.0022
Replace mentions	1,664	0.6320	-0.0015	0.5128	-0.0010
Demojize emoji	1,188	0.6204	-0.0131	0.4853	-0.0285
Unpack hashtags	2,359	0.6276	-0.0058	0.5029	-0.0109
Normalize Arabic characters	6,877	0.6330	-0.0004	0.5110	-0.0028
Remove diacritics	447	0.6345	+0.0011	0.5143	+0.0005
Remove tatweel	319	0.6338	+0.0003	0.5143	+0.0005
Normalize digits	1,275	0.6350	+0.0015	0.5190	+0.0052
Remove punctuation/symbols	7,110	0.6320	-0.0014	0.5130	-0.0008
Collapse repeated characters	507	0.6325	-0.0009	0.5095	-0.0043
Remove stop words	7,867	0.6345	+0.0011	0.5051	-0.0087
Lemmatize Arabic tokens	9,063	0.6307	-0.0027	0.5247	+0.0109
Preserve negation	1,817	0.6327	-0.0008	0.5151	+0.0013

Emoji demojization has the largest contribution: removing it lowers macro F1 by 0.0131 and POS F1 by 0.0285. Hashtag unpacking is the second-largest positive contributor. Both findings are consistent with sentiment being expressed through social-media conventions. Some removals slightly improve macro F1, notably disabling URL removal (+0.0046). This does not prove URLs are semantically desirable; it may reflect correlated domain markers. The effect is small and the default noise-removal policy is retained for consistency and generalization. Disabling lemmatization improves POS F1 but lowers macro F1, showing that a single aggregate can hide class-specific trade-offs.

4.4 Arabic data augmentation

Table 6: Validation effect of positive-class Arabic EDA.

Condition	Train rows	Acc.	Macro P	Macro R	Macro F1	NEG F1	NEU F1	POS F1
Baseline	9,273	0.6635	0.6410	0.6317	0.6335	0.7097	0.6769	0.5138
Arabic EDA	11,502	0.6671	0.6579	0.6112	0.6207	0.7114	0.6896	0.4612

EDA raises accuracy by 0.0036 but lowers macro F1 by 0.0127 and POS F1 by 0.0526. Positive precision rises from 0.5389 to 0.6196, while positive recall falls from 0.4909 to 0.3673. The synthetic transformations therefore make the classifier more conservative rather than improving minority coverage. Random swaps and deletions can preserve labels imperfectly and may create unnatural Arabic word order. The result argues against including this augmentation in the final baseline and motivates stronger contextual or back-translation methods in future work.

4.5 Class-imbalance handling

Table 7: Validation effect of balanced class weights on TF-IDF Random Forest.

Condition	Acc.	Macro P	Macro R	Macro F1	NEG F1	NEU F1	POS F1	POS recall
Unweighted	0.6370	0.6368	0.5772	0.5886	0.6564	0.6817	0.4275	0.3218
Balanced weights	0.6286	0.6039	0.6046	0.6037	0.6428	0.6704	0.4978	0.5036

Balanced weighting improves macro F1 by 0.0151, POS F1 by 0.0702, and POS recall by 0.1818, at the cost of 0.0084 accuracy and lower POS precision. This is the expected cost-sensitive trade-off: more minority examples are recovered while additional false positives are accepted. Unlike EDA, weighting changes the loss contribution without creating possibly unnatural text.

4.6 Influence of hyperparameter tuning

Table 8: Validation macro-F1 gain from the first configured trial to the selected trial.

Model	Trials	Selected trial	First-trial F1	Gain
TF-IDF + Multinomial NB	10	6	0.6089	+0.0246
Word2Vec + LSTM	4	3	0.6095	+0.0201
FastText + vanilla RNN	4	2	0.5974	+0.0175
Word2Vec + vanilla RNN	4	2	0.6140	+0.0038
FastText + Random Forest	8	3	0.5954	+0.0032
Word2Vec + Random Forest	8	5	0.5884	+0.0016
FastText + CNN	4	2	0.6323	+0.0012
Word2Vec + CNN	4	4	0.6247	+0.0009
Word2Vec + Gaussian NB	7	4	0.5273	+0.0006
FastText + Gaussian NB	7	1	0.5054	0.0000
FastText + LSTM	4	1	0.6164	0.0000
TF-IDF + Random Forest	8	1	0.5886	0.0000

Tuning matters most for Multinomial NB, Word2Vec LSTM, and FastText RNN. The best first-trial gains are 0.0246, 0.0201, and 0.0175. Several searches select the first candidate, which is still an informative result: performance is insensitive within the tested compact grid. The experiment avoids claiming that more search always improves a model.

5 Comparative Analysis and Discussion

5.1 Impact of representation choice

The best model in each representation family is FastText CNN (0.6380 test macro F1), TF-IDF Multinomial NB (0.6348), and Word2Vec LSTM (0.6262). FastText has the strongest single result because subword inference helps with Arabic morphology and unseen forms. However, averaging FastText vectors erases order and produces weak Gaussian NB performance. Representation quality therefore depends on how it is consumed, not only on the embedding algorithm.

TF-IDF is strongest when paired with its natural probabilistic classifier. Its explicit lexical and bigram evidence suits short tweets and requires no embedding training. Word2Vec performs best with LSTM, suggesting that sequence modeling recovers information lost by mean pooling. Its lack of subword inference is a limitation under dialectal and orthographic variation.

5.2 Traditional versus deep learning

The strongest neural model exceeds the strongest traditional model by only 0.0032 test macro F1. CNNs average 0.6317 test macro F1 across Word2Vec and FastText, while the four sequential neural models average 0.6186. Traditional models average 0.5847, but this average mixes the excellent TF-IDF NB result with weak dense Gaussian NB baselines. The evidence does not support a blanket claim that deep learning always wins; compatibility between representation and classifier is decisive.

Traditional models offer fast training, low memory use, and easier diagnosis. Neural models can use word order and local compositional patterns but require more computation, careful padding and length handling, early stopping, and seed-stability checks.

5.3 Non-sequential versus sequential neural models

The CNN is best for both embedding families except that Word2Vec LSTM narrowly exceeds Word2Vec CNN by 0.0008 test macro F1. Local n-gram detectors and global max pooling are well matched to short social-media text, where a small phrase, emoji neighborhood, or negated expression may determine sentiment. The vanilla RNN is consistently weaker than the corresponding CNN and LSTM. LSTM gating improves over vanilla RNN for both representations, but does not surpass FastText CNN.

5.4 Strengths and limitations of each combination

Table 9: Qualitative strengths and limitations of the evaluated combinations.

Combination	Strengths	Limitations
TF-IDF + NB	Fast, sparse, interpretable lexical baseline; near-best macro F1; strong NEG performance.	Ignores semantic similarity and most long-range order; depends on observed vocabulary.
TF-IDF + RF	Nonlinear interactions and robust ensemble.	Sparse high-dimensional splits are inefficient; weak POS recall without weighting.
Word2Vec + GNB	Compact dense input and very fast fitting.	Gaussian independence assumptions poorly match mean embeddings; word OOV problem.
Word2Vec + RF	Nonlinear dense baseline; high overall accuracy.	Mean pooling erases order; minority POS performance is weak.
Word2Vec + CNN	Strong local-pattern model; competitive accuracy and NEU F1.	Exact-vocabulary OOV handling and fixed sequence truncation.
Word2Vec + RNN	Explicit token order and compact architecture.	Ungated recurrence is seed-sensitive and weaker than CNN/LSTM.
Word2Vec + LSTM	Best Word2Vec model; gated memory; strong POS F1.	More compute and parameters; limited gain over simpler CNN.
FastText + GNB	Subword-aware vectors can represent unseen forms.	Mean Gaussian model underuses those vectors; lowest macro F1.
FastText + RF	Highest test accuracy and strong NEG/NEU F1.	Accuracy masks poor POS F1; mean pooling removes sequence information.
FastText + CNN	Best validation and test macro F1; subword coverage plus local phrase detectors.	Seed variance and lower POS performance than majority classes.
FastText + RNN	Subword-aware ordered sequence.	Vanilla recurrence is weaker than CNN and LSTM.

Combination	Strengths	Limitations
FastText + LSTM	Subword coverage and gated memory.	Does not translate extra complexity into the best score.

5.5 Sarcasm and dialect error analysis

Table 10: Selected-model test performance by sarcasm label.

Sarcasm	Rows	Accuracy	Macro F1	NEG F1	NEU F1	POS F1
False	2,504	0.6398	0.6158	0.5959	0.7126	0.5388
True	587	0.7853	0.4475	0.8775	0.2937	0.1714

Sarcastic accuracy is high because sarcastic records are strongly associated with NEG, but sarcastic macro F1 is low. The model performs poorly on sarcastic NEU and POS examples. This is a concrete example of accuracy hiding a class-conditional failure.

Table 11: Selected-model test performance by dialect.

Dialect	Rows	Accuracy	Macro F1	NEG F1	NEU F1	POS F1
Egyptian	610	0.6279	0.5542	0.7364	0.5000	0.4262
Gulf	171	0.6140	0.5929	0.6829	0.5586	0.5373
Levantine	123	0.6179	0.6206	0.6538	0.5301	0.6780
Maghrebi	8	0.5000	0.3111	0.6000	0.3333	0.0000
MSA	2,179	0.6861	0.6539	0.6893	0.7343	0.5382

MSA has the strongest well-supported macro F1. The apparent Levantine result is based on only 123 records, and the Maghrebi result on eight; neither should be generalized without confidence intervals or more data. Dialect is not used as a prediction feature, so this analysis diagnoses coverage rather than changing the model.

6 Challenges Encountered and Mitigation Strategies

1. **Duplicate leakage and conflicting annotation.** Simple random splitting would place equivalent tweets in different partitions. Conflicting sentiment labels also make deterministic collapse unsafe. Conflicting groups were removed, remaining duplicates collapsed, and complete processed-text groups assigned together.
2. **Arabic morphology and noisy social-media conventions.** Conservative normalization, lemmatization, protected negation, semantic emoji conversion, and hashtag retention were implemented as independent stages. Ablation quantified rather than assumed their value.
3. **Class imbalance.** Accuracy favored majority behavior. Macro F1 and per-class metrics were used, and augmentation and class weighting were evaluated independently. Class weighting improved minority recall; the tested EDA did not.
4. **Representation leakage.** TF-IDF vocabulary/IDF, Word2Vec, FastText, augmentation, and class weights were all fitted or derived from training data only. Held-out labels never

influenced representation learning.

5. **Fair model comparison.** Word2Vec and FastText neural architectures use matched sequence length, optimizer, batch size, grids, and frozen embeddings. Random Forest grids are held constant where possible. This makes differences more attributable to representations.
6. **Neural stochasticity.** Deterministic algorithms, fixed seeds, one-worker embedding training, stable hashing, fixed-epoch repeats, and three final seeds reduce and expose variability. The remaining seed spread is reported rather than hidden.
7. **Limited subgroup evidence.** Maghrebi and some dialect classes have very small support. Their results are labeled descriptive and no strong dialect conclusion is drawn.
8. **Compute and scope.** Compact systematic grids and early stopping make all mandatory experiments feasible. Optional transformer embeddings are deferred rather than reducing rigor on required representations and controlled studies.

7 Reproducibility and Implementation

The implementation uses Python 3.12, pandas and NumPy for data handling, NLTK and Qalsadi for Arabic preprocessing, scikit-learn for TF-IDF and traditional models, Gensim for Word2Vec and FastText, PyTorch for neural models, and Matplotlib/Seaborn for visualizations. Apple MPS is selected when available, with CPU fallback.

The notebook is the primary executable deliverable. A YAML configuration contains the split policy, preprocessing switches, feature settings, hyperparameter grids, controlled-experiment settings, and final seeds. Raw and processed data and model artifacts are regenerated locally. Machine-readable CSV and JSON outputs are the source of truth for report values. Notebook execution disables Jupyter timing metadata so repeat runs remain stable.

The complete workflow is:

1. verify and load pinned source files;
2. resolve raw duplicates and sentiment conflicts;
3. preprocess and create the grouped split;
4. conduct training-only exploratory analysis;
5. fit training-only representations;
6. tune every required classifier on validation macro F1;
7. run preprocessing, augmentation, and weighting controls;
8. freeze selections before test prediction;
9. evaluate all 12 combinations once on test data;
10. repeat only the selected winner under the prespecified seeds; and
11. generate final tables, figures, and error analyses from saved results.

8 Conclusion

The study completes every mandatory representation, classifier, controlled experiment, and evaluation requirement. FastText CNN is the validation-selected winner and reaches 0.6380 test macro

F1. Its advantage over TF-IDF Multinomial NB is small, so the main conclusion is not simply that a neural model wins. Rather, performance depends on a compatible pairing: sparse lexical evidence works very well with Multinomial NB, subword-aware sequences work well with CNN, and mean embeddings are not well served by the tested Gaussian NB assumption.

Emoji and hashtag processing are the most important observed preprocessing choices. Balanced class weighting provides a clear minority-class benefit, whereas random-swap/deletion EDA harms positive recall. Accuracy alone is misleading for FastText Random Forest and for sarcastic subgroups. The results therefore support macro and per-class reporting, strict leakage controls, and explicit stability analysis as essential parts of Arabic sentiment evaluation.

Future work should evaluate contextual Arabic transformer embeddings, contextual augmentation or back-translation, calibrated decision thresholds, and larger dialect-balanced samples. Those extensions should preserve the same grouped split and training-only fitting discipline.

A All Test Confusion Matrices

Table 12 reports every required test confusion matrix. Rows are actual classes and columns are predicted classes. The held-out test supports are NEG=1,249, NEU=1,293, and POS=549.

Table 12: Test confusion matrices for all 12 representation-model combinations.

Model	Actual	Pred. NEG	Pred. NEU	Pred. POS
FastText + CNN	NEG	945	227	77
	NEU	332	858	103
	POS	175	114	260
TF-IDF + Multinomial NB	NEG	968	202	79
	NEU	325	834	134
	POS	190	99	260
Word2Vec + LSTM	NEG	942	214	93
	NEU	351	822	120
	POS	170	117	262
Word2Vec + CNN	NEG	860	323	66
	NEU	248	958	87
	POS	151	169	229
Word2Vec + vanilla RNN	NEG	963	194	92
	NEU	383	772	138
	POS	185	104	260
FastText + LSTM	NEG	898	273	78
	NEU	317	881	95
	POS	162	150	237
FastText + vanilla RNN	NEG	917	251	81
	NEU	318	865	110
	POS	180	139	230
FastText + Random Forest	NEG	1,007	214	28

Model	Actual	Pred. NEG	Pred. NEU	Pred. POS
	NEU	350	898	45
	POS	235	146	168
	NEG	982	233	34
Word2Vec + Random Forest	NEU	343	895	55
	POS	250	140	159
	NEG	865	345	39
TF-IDF + Random Forest	NEU	288	958	47
	POS	193	183	173
	NEG	1,026	128	95
Word2Vec + Gaussian NB	NEU	609	550	134
	POS	281	50	218
	NEG	1,062	102	85
FastText + Gaussian NB	NEU	676	489	128
	POS	312	34	203
	NEG			

B Search Spaces

Table 13: Systematic validation search spaces.

Model	Search space
Multinomial NB	$\alpha \in \{0.01, 0.1, 0.5, 1, 2\}$; fit prior $\in \{\text{true}, \text{false}\}$
Gaussian NB	variance smoothing $\in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$
Random Forest	trees $\in \{200, 400\}$; depth $\in \{\text{none}, 40\}$; min leaf $\in \{1, 2\}$
CNN	filters $\in \{64, 128\}$; dropout $\in \{0.3, 0.5\}$; learning rate 10^{-3}
Vanilla RNN	hidden size $\in \{64, 128\}$; dropout $\in \{0.3, 0.5\}$; learning rate 10^{-3}
LSTM	hidden size $\in \{64, 128\}$; dropout $\in \{0.3, 0.5\}$; learning rate 10^{-3}

References

- [1] I. Abu Farha, W. Zaghouani, and W. Magdy, “Overview of the WANLP 2021 Shared Task on Sarcasm and Sentiment Detection in Arabic,” in *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pp. 296–305, 2021. <https://aclanthology.org/2021.wanlp-1.36/>
- [2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” *arXiv preprint arXiv:1301.3781*, 2013. <https://arxiv.org/abs/1301.3781>
- [3] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching Word Vectors with Subword Information,” *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017. <https://aclanthology.org/Q17-1010/>
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001. <https://doi.org/10.1023/A:1010933404324>

- [5] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” in *Proceedings of EMNLP*, pp. 1746–1751, 2014. <https://arxiv.org/abs/1408.5882>
- [6] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.